

Paruf	NAAC	Faculty Of Engineering & Technology Subject Name: High Performance Computing subject code: 303105356
University		

PRACTICAL-OS to check

Write a program task distribution using Gprof.

Aim: is a profiler

is a tool used to measure and analyse how different parts of a computer system resources, program particularly execution time, to identify performance within the code, or bottlenecks allowing developers to optimize specific sections of large-scale or better computing systems.

What

a "profiler" is
("hotspots")

performance reduction to gprof

Gprof is a performance analysis tool used to profile applications to determine where time is spent in program execution. Gprof is included with most Unix/Linux implementations, is easy to use, and can quickly worksw which parts of an application take the most time (hotspots). It works by instrumenting your code during compilation, and then sampling program execution during execution. Sampling data is saved in a file, gmon.out, which can later be read by the gprof command. Gprof was developed as an extension to the named gmon.out by providing a call graph of the program's execution.

of gprof

Feature Graph Generation:

1. Call Profile: Lists functions and Visualizes function relationships and call hierarchies.
2. Flat Profile: Shows execution time with execution time, helping identify time-consuming functions.
3. Cumulative Profile: Shows execution time measurement: functions Measures time spent in each function, including called functions.
4. Percentage of total time spent in a function and its descendants.
5. Percentage of total execution time for each function.
6. Multiple Runs Support: Aggregates profiling data from different program runs.
7. GCC Integration: Easily used with GCC by compiling with the -pg flag.
8. Human-Readable Output: Generates readable output for analysis.
9. Function Call Counts: Reports how many times each function is called.
10. Recursive Function Support: Accurately profiles recursive calls.

Parur
University



Faculty of Engineering & Technology
Subject Name: High Performance Computing
Subject Code: 303105356

Program 1

CODE:

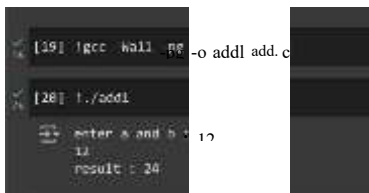
```
%%writefile add.c
#include<stdio.h>
int
add(int a, int b);
int
main() { int a,b;
printf("enter a and b :
scanf("%d" &a);
scanf("%d" &b); int result
— add(a,b); printf("result :
%d" ,result);

int add(int a , int b) {
return a+b;
```

gprof steps :

Step 1:

- Wall: Enables all warnings for better debugging.
- pg: Enables profiling by generating extra code to collect profiling information.
- addl: Specifies the output file as addl. containing the C code.
- add.c: The source file



Parur
University



Faculty of Engineering & Technology
Subject Name: High Performance Computing
Subject Code: 303105356

for analyzing program performance. file
to analyze.

Step 2 :

gprof: GNU profiler tool which profiles data generated from the program run. profiling
Specifies the executable to profile and the output file.
gmon.out: Contains the profiling data.
op.txt: Redirects the output to op.txt.

[21]

add addl add. c gmon . out sample

[23] ! gprof addl gmon.out > op. txt

Step 3 :

- Displays the content of the

op.txt file.

```

$ cat op.txt
Flat profile:
Each sample counts as 0.01 seconds.
no time accumulated

% cumulative self
time seconds seconds calls
0.00 0.00 0.00 1

```

```

self total
s/call ts/call name
0.00 0.00 add

```

Program 2

CODE:

```
%%writefile loop.c
```

```
#include<stdio.h>
```

```
int loop(int c); int
```

```
main() { int c; c =
```

```
10000; loop(c);
```

```
int loop(int c) { for(int i = 0;
```

```
    i < c; i++){ for(int j = 0; j < c;
```

```
        j++) {
```

Parur
University



Faculty of Engineering & Technology

Subject Name: High Performance Computing

Subject Code: 303105356

BTech-CSE 4th Year 7th Semester

```

999) {
    if(i ==9999      j
        printf("Loop ends\n"

return 0;

```

Gprof Steps:

Step 1

```

[53] gcc -Wall -pg -o loop loop.c
[54] ./loop
[55] Loop ends

```



Step 2

```

[56] ls add* addl* add. c gmon. out
[57] !gprof loop gmon.out> otp.txt

```



Step 3 :

```

[58] !cat otp.txt
Flat profile:
Each sample counts as 0.01 sec
% cumulative self
time seconds seconds c
100.00 0.20 0.20

```



Conclusion

Using gprof, we identified the time distribution across tasks. It highlights bottlenecks, helps excessive time, and ensures a balanced workload for better optimize functions consuming performance.